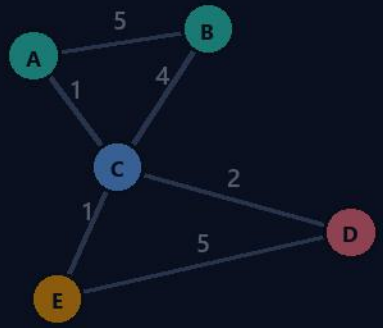
PARTITIONING FOR EDA · LAB 07

Recursive bisection

From one clean cut to four parts.

Reuse the bipartitioner you already trust.

```
one cut  →  four parts
```



A B | C | D | E

four parts, one cut at a time

PARTITIONING · LAB 07

The situation

you own a two-way cutter; the floorplan wants four blocks

WHAT YOU HAVE

One working bipartitioner

- spectral, Kernighan-Lin, or FM
- one clean cut you trust
- cut A B C | D E = 3 on the test graph

`ABC | DE  cut 3`

the cutter is not the problem

WHAT YOU NEED

Four connected blocks

- each block connected, sizes close
- no new four-way algorithm to write
- numbers you can still check by hand

`A B | C | D | E`

one more split than three parts

Recursive bisection reuses the cutter: the same two-way split, applied to smaller and smaller parts.

THE METHOD

The loop

one rule, repeated until the part count reaches k

PSEUDOCODE

```
parts = {all nodes}

while |parts| < k:

    P = largest part

    (P1, P2) = bipartition(P)

    replace P with P1, P2
```



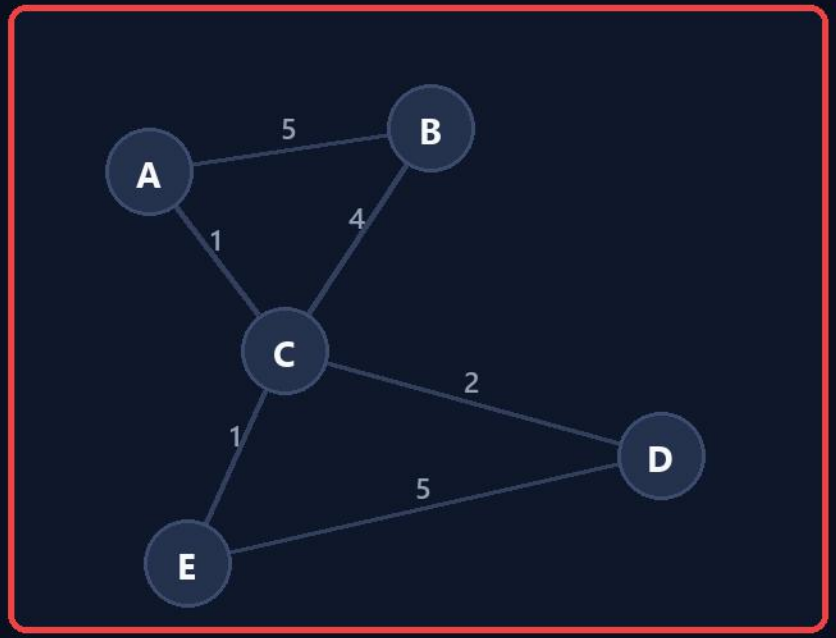
Each split is one ordinary bipartition of one part — the same spectral, KL, or FM cutter, on a smaller graph.

balance comes from choosing the largest part first

THE TEST GRAPH

Five nodes, six weighted edges

small enough to check every cut by hand



EDGE WEIGHTS

A – B	5	heavy
B – C	4	heavy
D – E	5	heavy
C – D	2	light
A – C	1	light
C – E	1	light

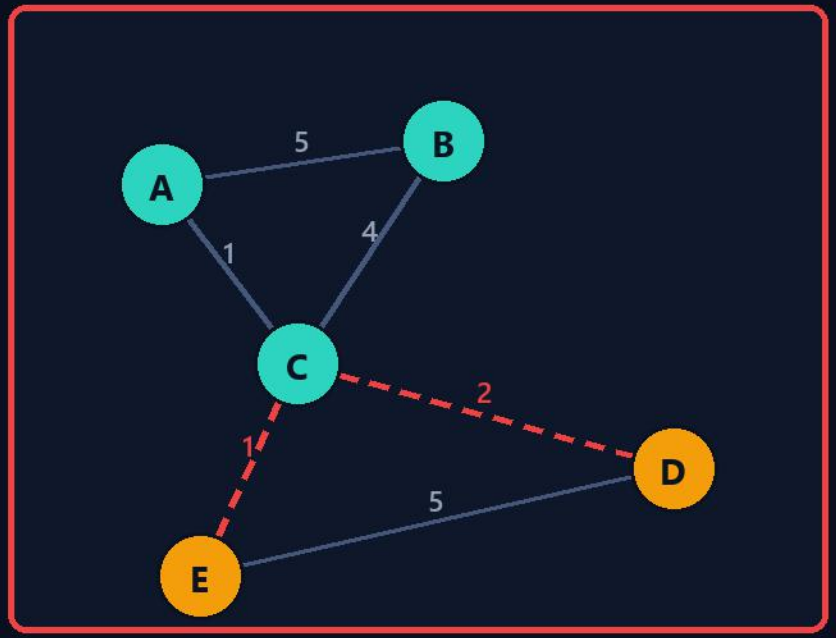
cut = total weight of edges whose ends land in different parts

target: four parts

THE ALGORITHM, ON THE TEST GRAPH

First split: A B C | D E

the cutter works on the whole graph first



PARTS

ABC|DE

2 of 4 parts placed

CROSSING EDGES

C – D	2
C – E	1

CUT

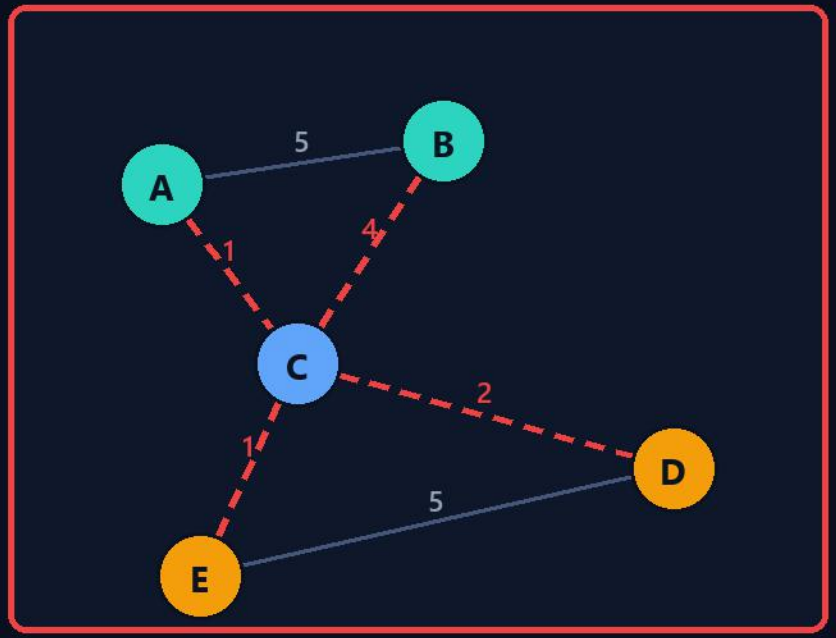
3

2 + 1 = 3

THE ALGORITHM, ON THE TEST GRAPH

Second split: A B | C | D E

split the largest part only — inside A B C



PARTS

AB|C|DE

3 of 4 parts placed

CROSSING EDGES

A – C	1
B – C	4
C – D	2
C – E	1

CUT

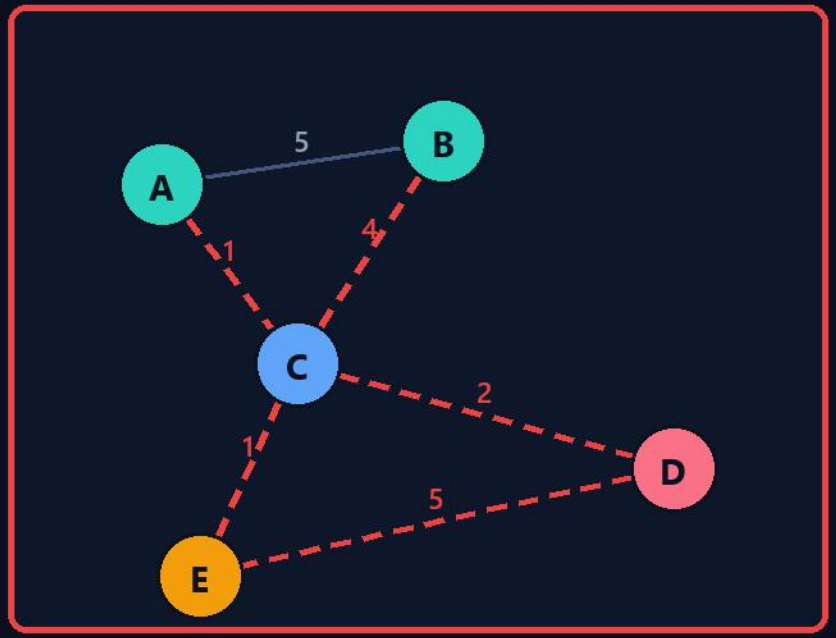
8

3 + 5 = 8

THE ALGORITHM, ON THE TEST GRAPH

Third split: D and E part

the heavy weight-five edge has to be cut



PARTS

AB|C|D|E

4 of 4 parts placed

CROSSING EDGES

A – C	1
B – C	4
C – D	2
C – E	1
D – E	5

CUT

13

8 + 5 = 13

HANDS-ON

Read the panel, then run it

the lab opens from the button under this clip

Recursive bisection

starter graph · target k = 4 · example run

Run k=3 | Run k=4

metrics

```
targetK: 4
step 1: parts=ABC|DE cut=3
step 2: split=ABC parts=AB|C|DE cut=8
step 3: split=DE parts=AB|C|D|E cut=13
```

Challenges

2 / 10 cleared

```
goldens: AB|C|DE
cut 8
```

Check

parts

grouping by node; the bars are part boundaries

cut

total weight crossing every boundary

history

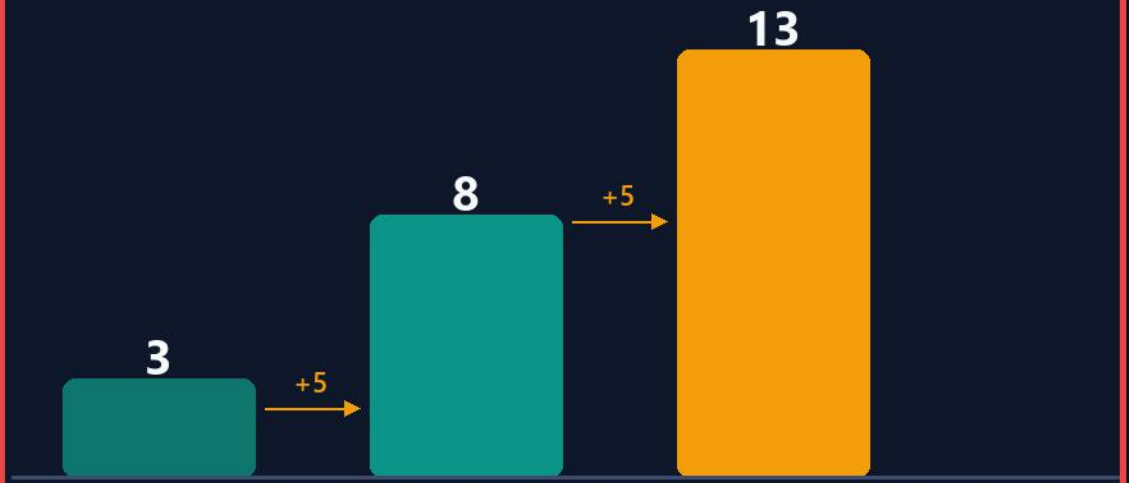
which part was split, and what it became

if the numbers disagree, the history shows where

THE TRADEOFF

More parts, more cut

the last split cuts the heaviest edge in the graph



total cut

`k = 3 stops at 8`

`the fourth part costs +5 (D–E)`

GREEDY

Each split is locally good. An early cut can leave the next one with no good options.

fewer splits can mean
a lower total cut

`k=3 → 8` vs `k=4 → 13`

the cost is the edge you are forced to cut

PITFALLS

Three traps

each one is cheap to check before you claim an answer

01 A cheap cut, unbalanced blocks

One node on one side and four on the other can look cheap, and still be useless: the blocks are wildly unbalanced.

02 Half a report

Report total cut and part sizes together; either number alone hides the other.

03 Moving pinned nodes

If a node is pinned to a side, the split must respect it; a lower cut from moving it is not a valid answer.

SELF-CHECK

Three questions

answer them before you touch the lab

1. After the first split, why cut A B C next — and not D E?
2. Why does the cut jump from 8 to 13 when D E is split?
3. For exactly three parts, where do you stop, and what is the cut?

IF ALL THREE

are easy,
you own
this loop.

if not,
replay
splits 1–3.

YOUR TURN

Run it, then break it

the lab opens from the button under this clip

YOUR CHECKLIST

1. Load the starter graph.
2. Press Run k=3, read the history.
3. Press Run k=4; compare cut 13 with 8.
4. Clear the challenge cards until Check is green.
5. Then the quiz, then the next lab.

Run k=3 | Run k=4 | Check

NEXT

Multiway partitioning: stop splitting one part at a time and go straight to k.

goldens

k=3 → 8

k=4 → 13

track A: implement the loop on the tiny graph

track B: the browser lab